2026-06-04 개발 일지

> 대상 작업: Phase 2(팀 빌딩) UI 전면 개편 / Phase 3(전술 배치도) 선수 자리 교체 / 다크모드 차단 관련 커밋: `9e94188`(팀 구성 페이지 UI 전면 개편 + 다크모드 차단), `1107c36`(전술 배치도 자리 교체)

1. 개발 일지

학습한 내용 및 활동 결과

오늘은 팀 구성 페이지(`TeamPage`, `TeamBuilder`)를 기능 위주의 단순 목록에서 **데이터 시각화 중심의 대시보드 UI**로 전면 개편했다. 먼저 Snake Draft 알고리즘이 반환하던 `metrics`(밸런스 점수, 실력 차이, 팀별 평균 나이, 나이 페널티)를 `StatCard` 컴포넌트로 분리해 한눈에 비교할 수 있도록 했고, 실력 차이 값에 따라 초록/주황/빨강으로 색이 바뀌는 조건부 강조(accent)를 적용해 균형 정도를 직관적으로 인지하도록 만들었다. 선수 행은 포지션별 컬러 배지(GK/DF/MF/FW)와 5점 만점의 dot 게이지(`SkillDots`)로 재설계했다. 또한 React에서 자주 쓰는 `useMemo` 의존성 배열을 활용해 가중치·반복 횟수·재배정 시드(`refreshSeed`)가 바뀔 때만 알고리즘을 다시 실행하도록 최적화했다. 사용성 측면에서는 브라우저 기본 `alert`를 3초 후 자동으로 사라지는 인라인 토스트(`savedMsg`)로 교체했고, 경기 선택을 커스텀 드롭다운으로 바꿨다. 마지막으로 `dnd-kit` 기반 전술 배치도에 **선수 자리 교체(swap)** 로직을 추가해, 드롭 위치 9% 반경 안에 같은 팀 선수가 있으면 두 선수의 좌표가 자동으로 맞바뀌도록 구현했다.

고찰 및 추후 개발 계획

기능 구현에 집중하던 화면을 시각 정보가 풍부한 대시보드로 바꾸니 총무가 팀 균형을 즉시 판단할 수 있게 되어, 프로젝트의 핵심 목표인 "객관적·공정한 팀 배정"의 체감도가 크게 올라갔다. 다만 밸런스 지표가 다섯 개로 늘면서 모바일 화면에서는 정보 과밀이 우려되므로, 다음에는 핵심 지표 2개만 노출하고 나머지는 펼침(accordion) 처리할 계획이다. 또한 전술 배치도의 자리 교체 임계값(9%)이 고정값이라 화면 크기에 따라 체감이 달라지므로, 필드 크기에 비례하는 상대값으로 리팩터링하고 교체 시 짧은 애니메이션 피드백을 추가할 예정이다.

소스코드/실행화면 캡처

캡처 1 — 밸런스 지표 StatCard + 조건부 색상 강조 (`src/components/TeamBuilder.jsx`)

```
{/* 밸런스 지표 */}
<div className="tb-metrics">
  <StatCard label="밸런스 점수" value={result.metrics.score.toFixed(2)} />
  <StatCard
    label="실력 차이"
    value={result.metrics.skillDiff.toFixed(2)}
    accent={result.metrics.skillDiff < 1 ? '#2ecc71'
          : result.metrics.skillDiff < 2 ? '#f39c12' : '#e74c3c'}
  />
  <StatCard label="팀 A 평균 나이" value=
{`${result.metrics.avgYearA.toFixed(1)}세`} />
  <StatCard label="팀 B 평균 나이" value=
{`${result.metrics.avgYearB.toFixed(1)}세`} />
  <StatCard label="나이 페널티" value={result.metrics.yearPenalty.toFixed(2)}
```

```
/>
</div>
```

캡처 2 — 포지션 컬러 배지 + 실력 dot 시각화 (`src/components/TeamBuilder.jsx`)

```
const POSITION_COLOR = {
  GK: { bg: '#fff3cd', color: '#856404' },
  DF: { bg: '#cfe2ff', color: '#084298' },
  MF: { bg: '#d1e7dd', color: '#0a3622' },
  FW: { bg: '#f8d7da', color: '#842029' },
};

const SkillDots = ({ score, max = 5 }) => (
  <div className="tb-skill-dots">
    {Array.from({ length: max }, (_, i) => (
      <span key={i} className={`tb-skill-dot ${i < score ? 'is-filled' :
''}`} />
    ))}
    <span className="tb-skill-num">{score}</span>
  </div>
);
```

캡처 3 — 재배정 시드/가중치 변경 시에만 재계산하는 useMemo (`src/components/TeamBuilder.jsx`)

```
const result = useMemo(() => {
  if (normalizedParticipants.length < 2) return null;
  return buildBalancedTeams(normalizedParticipants, {
    yearWeight: ageWeight,
    iterations,
    seed: refreshSeed,
  });
}, [normalizedParticipants, ageWeight, iterations, refreshSeed]);
```

캡처 4 — 전술 배치도 자리 교체 로직 (`src/components/TacticsBoard.jsx`)

```
// 드롭 위치 근처(9% 이내)에 같은 팀 필드 선수가 있으면 자리 교체
const SWAP_THRESHOLD = 9;
const nearbyPlayerId = Object.entries(prev).find(([pid, placement]) => {
  if (pid === activePlayerId) return false;
  if (playerTeamMap[pid] !== playerTeam) return false;
  if (placement?.area !== 'field') return false;
  return Math.abs(placement.x - x) < SWAP_THRESHOLD
      && Math.abs(placement.y - y) < SWAP_THRESHOLD;
})?.[0];
```

> 📸 실행화면 캡처는 `npm run dev` 실행 후 `/?page=teams`(팀 구성)와 전술 배치도 화면을 첨부할 것.

2. 이슈 발생 처리

이슈 ① OS 다크 모드에서 화면이 깨지는 문제

발생한 이슈 사용자 기기의 OS가 다크 모드일 때, 별도의 다크 테마를 구현하지 않았음에도 브라우저가 폼 입력창·드롭다운 등 기본 UI 컨트롤의 색을 임의로 반전시켜, 흰 배경 위 흰 글씨처럼 가독성이 무너지는 현상이 발생했다.

해결 방법 원인은 브라우저가 `prefers-color-scheme: dark`를 감지하면 폼 요소와 시스템 색상을 자동으로 다크 팔레트로 렌더링하기 때문이었다. EPL은 라이트 테마 단일 디자인이므로, 루트 스타일(`src/styles/variables.css`)의 `:root`에 `color-scheme: light;`를 선언하고 `index.html`의 `<head>`에도 `<meta name="color-scheme" content="light" />`를 추가했다. 이렇게 CSS와 HTML 양쪽에서 라이트 스킴을 명시하면 브라우저가 OS 설정과 무관하게 라이트 색상으로 폼 컨트롤을 렌더링한다. 메타 태그를 함께 둔 이유는 CSS 로드 이전 초기 페인트 단계에서도 다크 반전이 일어나지 않도록 보장하기 위함이다.

```
:root {
  color-scheme: light;   /* OS 다크모드 차단 */
  --color-primary: #2ecc71;
  ...
}
```

이슈 ② 벤치 선수를 필드로 드롭할 때 기존 선수가 사라지는 문제

발생한 이슈 전술 배치도에서 벤치에 있던 선수를 이미 다른 선수가 점유한 필드 위치로 드래그하면, 두 선수가 같은 좌표에 겹쳐 한 명이 보이지 않게 되는 문제가 있었다.

해결 방법 처음에는 드래그한 선수의 좌표만 갱신했기 때문에 기존 선수가 그대로 남아 겹쳤다. 이를 해결하기 위해 좌표를 덮어쓰기 전에 같은 팀의 필드 선수 중 드롭 위치 9% 반경 안에 있는 대상을 먼저 탐색하고, 대상이 있으면 두 선수의 위치를 **맞교환**하도록 변경했다. 특히 드래그한 선수의 직전 상태(`activePrev`)가 벤치였는지 필드였는지를 분기 처리해, 벤치 출신이면 교체 대상 선수를 원래 벤치 슬롯(`benchSlotId`)으로 되돌리고, 필드 출신이면 좌표·포지션 정보를 그대로 넘겨주도록 했다. 또한 부분 갱신으로 인한 상태 누락을 막기 위해 매 드롭마다 전체 선수의 배치 상태를 `prev` 객체로 재구성한 뒤 한 번에 `setPlacementOverrides`로 반영해 일관성을 확보했다.

```
// 기존 선수를 드래그한 선수의 원래 위치로 (벤치/필드 분기)
next[nearbyPlayerId] = activePrev?.area === 'bench'
  ? { area: 'bench', benchSlotId: activePrev.benchSlotId }
  : { area: 'field', x: activePrev.x, y: activePrev.y,
      positionId: activePrev.positionId, positionLabel:
activePrev.positionLabel };
```